CTC41 - Compiladores Lab

Prof. Luiz Mirisola - 2026

August 12, 2026

1 Como comparar - guia para correção

1.1 lib/log.c

No arquivo `lib/log.c`, há funções e váriaveis globais implementando um logger que imprime a saída de cada parte da compilação em um arquivo diferente. Leia o manual `doxygen` do diretório `lib`, que inclui exemplos de uso de cada função. Já estão implementadas no código fornecido do TinyCompiler.

> Nunca utilizem `printf` ou `fprintf`, substituam pelas funções do logger, para que cada saída seja enviada para o arquivo correspondente à fase correta da compilação, e para outro arquivo que armazena as saídas que indicam erro (não é o `stderr`, é outro arquivo).

1. No `main.c`, são tratados parâmetros, e o logger é inicializado com a função `initializePrinter()`.
   (a) Na inicialização, a fase de compilação corrente é definida como analise léxica (`LEX`).
2. funções `doneLEXstartSYN()` e similares, mudam a fase de compilação corrente. O código fornecido já chama as funções necessárias entre a execução de cada fase de compilação.
3. a função `pc()`[1] - print current, imprime na saída da fase de compilação corrente, **E** no stdout.
4. a função `pce()` - print current error, imprime na saída da fase de compilação corrente, no `stdout`, **E** na saída de erro.
   (a) As duas funções anteriores já são usadas no código fornecido (`fprintf` será usado apenas no `main.c`), e cobrem o caso desta disciplina, em que as fases de compilação são bem separadas e basta mudar a fase corrente depois da execução de cada uma.
5. a função `pp()` permite escolher, hardcoded, em quais saídas imprimir. Este controle fino pode ser usado em casos não previstos, fora do padrão. Constantes bitmask definem em quais arquivos a string será impressa.
   (a) verifique o `enum FileDestination` para os nomes das constantes bitmasks.
   (b) bitmasks podem ser combinadas com a operação OU bit-a-bit, o operador '|'[2].

1.2 diretórios output e alunoout

São gerados arquivos `txt` com toda a saída de todas as fases da compilação.

1. no diretório `alunoout`, serão armazenados as saídas do aluno e arquivos `.diff` indicando as diferenças entre as saídas do aluno e do gabarito, para todas as fases da compilação.
2. o diretório `output` contém as saídas do gabatiro, inclusive a tabela de símbolos e geração de código intermediário, logo, será diferente da saída de um compilador incompleto.
   (a) em `main.c` há #`ifdefs` que permitem definir quais fases da compilação serão executadas.

[1] no código há um #define para o program counter, que no código original do livro se chamava `pc`. Renomeei-o para evitar conflito de nomes.

[2] Não confunda com o OU lógico: operador '||'

1.3 Comparando diretórios detail x alunodetail

Estas são as ferramentas principais de avaliação das saídas para as fases Análise Sintática e subsequentes. Para a Análise Léxica, análises similares se aplicam em outros diretórios, leia a seção 2.

Os diretórios `detail` e `alunodetail` contém um `txt` para cada fase da compilação, para cada arquivo de teste, respectivamente do gabarito, e do aluno.

1. `make ddiff` faz diff com os arquivos em `detail` (gabarito) contra `alunodetail` (aluno), separando por fase da compilação, e armazenando-os em `alunodetail`.
2. `batcat *err*diff` imprime todos os diffs de erro de uma vez, coloridos e separados por arquivo. Leia §7.2.
   (a) outro exemplo: `batcat *syn*txt` imprime todas as saídas da fase de análise sintática, separadas por aquivo.
3. `meld alunodetail/ detail/` permite comparar os diretórios inteiros com código de cores indicando arquivos modificados ou faltantes, e ao clicar em cada arquivo, as diferenças são destacadas. Leia §7.3.

As saídas das fases posteriores já presentes em `detail` permitem adiantar as entregas, e podem servir como referência para estudo e entendimento dos exemplos.

2 Análise Léxica

Esta fase é a única onde é exigido diff zero. Acertar o formato das mensagens da saída faz parte do trabalho.

Existe um diretório especial, chamado `detailonlylex` para as saídas da análise léxica e de erro da primeira fase da implementação, que apenas precisa implementar a análise léxica.

O target `lexdiff` compara as saídas léxica e de erro do aluno contra o diretório `detailonlylex`, é esta comparação que deve chegar em diff zero.

O comando já está embutido no script de compilaçaõ descrito na seção 6.2.1, mas pode ser chamado diretamente:

```
~/ctc41/build$ make lexdiff
```

É um target do Makefile, portanto, deve ser executado depois do cmake, e no diretório onde está o Makefile gerado pelo cmake, ou seja, no diretorio `build`.

- Troubleshooting: Logo no início da execução do cmake, o código abaixo em `CMakeLists.txt` imprime uma mensagem indicando onde a saída do Flex será gerada. Se esta mensagem estiver vazia, indica que o Flex não foi encontrado pelo cmake.
  - `message(" * Flex OUT = ${FLEX_scanner_OUTPUTS}") )`

2.1 Porque a análise léxica precisa ser tratada em separado

A análise léxica não constrói árvore sintática, e não pára a análise do código em caso de erro. Inclusive porque não é capaz de detectar a maioria dos erros, pois tratar erros posteriormente é mais padronizado e manutenível. Alguns erros serão detectados, correspondendo à caracteres inválidos, mas mesmo assim a analise léxica continuará.

Por isso, as saídas léxica e de erro nas fases posteriores serão diferentes: pararão depois dos primeiros erros e mostrarão mensagens de erro mais detalhadas.

O compilador Tiny-C no diretório `TinyFlex` realiza apenas a análise léxica pode ser adaptado para compilar Cminus até passar na análise léxica. Mas, quando implementarem as próximas fases da compilação, terão que seguir as instruções da seção 3.1.

3 Análise Sintática

O diff das fases anteriores será sempre exigido, inclusive por ajudar a detectar bugs durante o desenvolvimento. `make ddiff` já compara todas as fases em separado, e **não deve ser abandonado** em favor de scripts que comparem apenas a fase corrente ou que comparem apenas a saída toda e não cada fase em separado.

- Podem seguir as mesmas convenções da saída fornecida, e atingir diff zero, e assim correção trivial.
- **ou**, podem seguir convenções diferentes, **mas**:
  - a sintaxe deve ser a mesma. Onde a sua árvore for igual à do gabarito, o formato do texto gerado para representar a árvore deve ser o mesmo.
  - a sua árvore sintática deve indicar uma árvore sintática válida.
  - deve haver identação apropriada para indicar os níveis da árvore, conforme as mesmas convenções do gabarito.
  - a sua saída deve refletir coerentemente as suas convenções.
  - devem ter consciência sobre o porque das diferenças e saber explicar o que fizeram diferente.
- Erros: há exemplos de erros detectáveis na Análise Sintática, e outros não-detectáveis, ou seja, que só serão detectados depois de construir a tabela de símbolos (os arquivos com prefixo `ser_`).
  - Mesmo para erros detectáveis, podem seguir convenções diferentes sobre quando parar a compilação, e portanto parar o output em momentos diferentes. **O importante é detectar o primeiro erro corretamente e imprimir uma mensagem correta**.
    - recomendo que parem a compilação depois do primeiro erro, porque: dependendo do erro, a) continuar a compilação apenas incorre em mais erros espúrios, e b) aumenta a probabilidade de crashes.
  - Para erros não detectáveis, haverá diferenças nas saídas, e é preciso **saber explicar porque**. Por exemplo, um erro de tipo só é detectado depois de construir a tabela de símbolos.
  - **Depois** de implementar a tabela de símbolos, na terceira etapa, todos os erros `compile time` dos exemplos precisam ser detectados.
  - É necessário imprimir uma mensagem de erro. **Não é permitido parar a compilação devido a um erro sem imprimir uma mensagem de erro.**
  - A mensagem de erro precisa ser no mínimo tão específica quanto à mensagem de erro correspondente na saída gabarito fornecida. Pode ser **mais** específica, não menos. Presentemente, este requisito é fácil pois as mensagens impressas são inespecíficas.

3.1 Transição da etapa análise léxica para Análise Sintática

1. Leia a seção 6.2 para entender o que muda na compilação quando a variável cmake `DOPARSE` é setada como TRUE. Note que não é preciso mudar o CMakeLists.txt, basta mudar de `FALSE` para `TRUE` o parametro na linha de chamada do cmake.
2. Leia a seção 1.3, que detalha as ferramentas de comparação das saídas para as próximas fases.
3. Devem passar a se basear no compilador Tiny-C do diretorio `TinyGeracaoCodigo`. Há incompatibilidades:
   (a) em `main.c`, a biblioteca de impressão é inicializada com a bitmask `LOGALL`, para gerar as saídas de todas as fases da compilação. Na versão `TinyFlex`, ela é inicializada com a bitmask `LER`, correspondendo apenas às saídas da análise léxica e de erro.
   (b) Em `TinyFlex/globals.h`, há um enum chamado `TokenType`, que não existe em `TinyGeracaoCodigo/globals.h`. O bison implementa as definições necessárias, este enum apenas foi definido manualmente devido à não utilização do bison. Com o bison, os tokens serão definidos nos arquivos `.l` e `.y`.

(c) Em `TinyGeracaoCodigo/main.c`, o `#define NOPARSE` é `FALSE`, para executar o trecho de código referente à Análise Sintática. Os defines `NO_ANALYSE` e `NO_CODE`, referentes às próximas etapas, respectivamente, Tabela de Símbolos e Geração de Código, estão definidos como `FALSE`, mas deverão ser setados como TRUE enquanto as etapas correspondentes não são implementadas.

(d) Dica: compare os diretórios `TinyFlex` e `TinyGeracaoCodigo` para visualizar as diferenças acima rapidamente:

i.

```
~/gitctc41/gen$ meld TinyFlex TinyGeracaoCodigo
```

4. É imprescindível usar a lib de log (`pc(), pce()`), e não usar `fprintf` diretamente. `TinyGeracaoCodigo/main.c`, já inclui as chamadas `doneXXXstartYYY()` para mudar o estado da lib de log, separando as saídas de cada etapa da compilação nos seus arquivos correspondentes.

5. Para a Análise Sintática, são relevantes as saídas `LEX`, `SYN`, e `ERR`.

6. Troubleshooting: Logo no início da execução do cmake, o código em `CMakeLists.txt` imprime uma mensagem indicando onde a saída do Flex será gerada. (Comando `message(" * Flex OUT = ${FLEX_scanner_OUTPUTS}")`). Se esta mensagem estiver vazia, indica que o *Flex não foi encontrado* pelo cmake. Analogamente, se a variável `DOPARSE` for `TRUE`, outra mensagem será impressa indicando onde a saída do Bison será gerada. Se esta mensagem for vazia, o *Bison não foi encontrado* pelo cmake.

3.2 Gramática aumentada e exemplos block2/3

Deverão modificar a gramática do C- dada no livro para incluir as funcionalidades dos exemplos `block2.cm`, `block3.cm`, e `mdc2.cm` cuidando de manter a gramática não-ambígua para evitar bugs:

1. Permitir intercalar declarações e comandos (veja as variáveis `a` e `b` em `block2.cm`).

2. Permitir declarar blocos de escopo aninhados com a sintaxe `{ }`, e construir uma pilha de escopos apropriada. (ver função `teste` de `block2.cm`, que possui um escopo interno).

3. Operador de subtração unário. (veja `block3.cm` e `mdc2.cm`)

2026: o gabarito apresenta bugs ao gerar código para calcular o resultado de expressões com o operador unário, que devem ser sanados no código de vocês.

4 Tabela de Simbolos

O diff das fases anteriores será sempre exigido, inclusive por ajudar a detectar bugs durante o desenvolvimento. `make ddiff` já compara todas as fases em separado, e **não deve ser abandonado** em favor de scripts que comparem apenas a fase corrente ou que comparem apenas a saída toda e não cada fase em separado.

- Em linhas gerais, esta fase envolve **dois** percursos na árvore sintática. No primeiro percurso, a tabela é construída, e alguns erros são detectados. O segundo percurso é necessário pois alguns erros só podem ser detectados depois que a tabela estiver completamente construída.

- É preciso repetir o formato de linha da saída fornecida. **SE** seguirem a mesma convenção de ordem das variáveis na tabela, é possível atingir diff zero, mas não é necessário.

- Diferentes estratégias podem resultar em ordem diferente das variáveis na tabela, e portanto das linhas da saída da tabela de símbolos: o programa python `compare_diffs.py` coloca as linhas em sets, e compara os sets (Veja seção 7.1). Um resultado `OK` indica que as mesmas strings aparecem em ordem diferente, e é suficiente.

  – O programa python compara os sets com `==`, implicando em comparar strings com o operador usual. Não pretendo aceitar comparações excluindo espaços, pois nesse caso as saídas dos alunos poderiam não ser formatadas como tabela.

  - A função `pc()` é apenas uma camada encapsulando `fprintf`. Opções do `printf` relacionadas a imprimir strings de tamanho constante continuam válidas.

- A coluna Scope deve diferenciar variáveis com o mesmo nome em diferentes escopos.

- O escopo referente a blocos definidos por `{ }` independentes de condicionais, funções ou loops podem ser nomeados usando o número de linha do ínicio do bloco como sufixo aplicado ao nome do escopo principal, normalmente um nome de função. Por exemplo, `teste:5` em `block2_tab.txt`, correspondendo ao escopo do bloco iniciado na linha 5, dentro da função `teste`.

  - aceito outras opções de denominação que diferenciem claramente cada bloco.

- Todos os (primeiros) erros devem ser detectados na saída de erro, inclusive os erros relacionados a tipos.

  - Alguns erros podem ser detectados no primeiro percurso, sendo possível escrever suas mensagens de erro na saída TAB antes da tabela. Outros erros só podem ser detectados no segundo percurso, sendo normalmente impressos depois da tabela. O script de comparação de set de strings deve continuar indicando OK caso a implementação do aluno resulte em uma diferente ordem de impressão entre tabela e mensagem de erro. Está bem, desde que a mensagem de erro continue correta. Note que a saída ERR não é afetada por esse problema, pois ela contém apenas a mensagem de erro.

- As funções `input` e `output` podem ser tratadas como pré-existentes, como se fossem definidas pela lib padrão da linguagem, e tendo identificadores já definidos no escopo global, o mesmo do main, denotado pela string vazia na tabela.

  - Implica que elas podem aparecer na tabela mesmo em exemplos em que nunca aparecem no código, como se fossem sempre carregadas e disponibilizadas pela linguagem.
  - Esta solução me parece fácil. Mas podem implementar diferente. Diffs apenas envolvendo estas funções serão aceitos, se souberem explicar como implementaram.
  - Caso seja pedido implementar `#includes`, esta orientação fica invalidada.

Alguns erros semânticos não seriam erros na linguagem C, ou seja, o código errado compilaria e executaria com um compilador C usual, sem nenhuma mensagem de erro. Isso é feature, não bug. A linguagem C- é menos flexível, tornando a geração de código mais fácil de implementar, com menos casos especiais, e a linguagem mais previsível e legível. Implementar um compilador C completo seria bem mais difícil!

5 Geração de Código

O diff das fases anteriores será sempre exigido, inclusive por ajudar a detectar bugs durante o desenvolvimento. `make ddiff` já compara todas as fases em separado, e **não deve ser abandonado** em favor de scripts que comparem apenas a fase corrente ou que comparem apenas a saída toda e não cada fase em separado.

- A comparação com diff é **irrelevante** para a fase de geração de código. *O importante é o seu código compilado executar corretamente para os exemplos completos de código funcional.*

  - A saída do gabarito e a comparação com diff *pode* ser útil como exemplo ou debug.
  - É possível adotar convenções diferentes para a geração do código que resultem em código diferente. Não é problema, desde que a execução seja correta.
  - O gabarito pode não implementar todas as funcionalidades pedidas neste ano. Um motivo adicional para não tentar gerar código igual ao gabarito.

- O compilador precisa detectar os erros em códigos com erro, e não é necessário gerar código executável para eles.

- Funções `input` e `output`: o simulador possui instruções `IN` e `OUT` para implementá-las.

- Algumas turmas anteriores compilaram para código intermediário de 3 endereços. Não é o mesmo pseudo-assembly do simulador `tm`.

5.1 O simulador tm

1. O simulador `tm.c` é compilado simplesmente com `gcc -o tm tm.c`
2. A entrada deve ser o código gerado pelo seu compilador, gerados em `alunodetail` com o sufixo `gen.tm`. Exemplos gerados pelo gabarito estão disponíveis em `detail` com o sufixo `gen.tm`
3. executando-se `tm`, use a opção `-h` para descobrir como executar o código.
   (a) o comando `g` (go) inicia a execução do código.
   (b) o comando `c` (clear) reseta a simulação, permitindo executar outras vezes sem chamar o simulador novamente.
   (c) outras opções podem ser úteis para debug.

6 Ferramentas cmake e compilação

6.1 Conceito: make targets

Um makefile pode ter vários *targets*. Cada target corresponde à execução de uma sequência de comandos, normalmente para compilar alguma coisa, ou executar scripts. Pode incluir a execução de outros targets (dependências).

Alguns targets sempre são executados, inclusive por default, chamando o make sem argumentos. Outros targets só são executados se o nome do target for um argumento na chamada do make, na forma `make <nome_do_target>`.

Ao se utilizar cmake, targets são definidos no arquivo `CMakeLists.txt`. É um target do Makefile, portanto, deve ser executado onde está o Makefile gerado pelo cmake, ou seja, no diretorio `build`.

Uma vez compilado, ainda no diretório `build`, o makefile gerado ainda pode ser utilizado para executar outros targets.

6.2 compLabAluno.bash (script de compilação)

```
1 cmake ..  -DDOPARSE=TRUE
2 make ddiff VERBOSE=1
```

O script deve ser chamado no diretório build, prefixado com "`..`" pois ele está no diretório pai:

```
1 me@mypc:~/ctc41/gen/build$ ../compLabAluno.bash
```

A primeira linha chama o cmake:
1. "`..`" implica em utilizar as instruções do diretório pai do corrente.
2. `-D` significa setar um parâmetro no cache do cmake. (veja `man cmake`).

O parâmetro em questão é chamado `DOPARSE`. `TRUE` significa que:

1. bison será incluído como dependência, e será chamado.
   (a) Logo no início da execução do cmake, o código em CMakeLists.txt imprime uma mensagem indicando onde a saída do Flex será gerada. (Comando `message(" * Flex OUT = ${FLEX_scanner_OUTPUTS}")`). Se esta mensagem estiver vazia, indica que o Flex não foi encontrado pelo cmake. Analogamente, se o parâmetro `DOPARSE` for TRUE, outra mensagem será impressa indicando onde a saída do Bison será gerada. Da mesma forma, se esta mensagem for vazia, o Bison não foi encontrado pelo cmake.
2. o diretório `detail` será usado como gabarito, e não `detailonlylex`.
3. O compilador Tiny-C a ser compilado será o compilador em `TinyGeracaoCodigo`, e não em `TinyFlex`
   (a) o executável `build/tiny` agora é um compilador completo. Ao compilar o exemplo `.ty`, as saídas de todas as etapas serão impressas.

(b) o executavel `build/tiny` loopa ao tentar compilar alguns exemplos de código Cminus (lembrem, é um compilador para TinyC). Portanto, se todo o código de `TinyGeracaoCodigo` for copiado no diretório `src`, o executável do compilador será gerado, mas a compilação dos exemplos não terminará.

Setar para `FALSE` implica no contrário. Para a análise léxica, sete para `FALSE`, para fases posteriores, sete para `TRUE`.

A segunda linha chama o make, executando o `Makefile` criado pelo cmake no diretório `build`.

1. `ddiff` é um target. Veja 6.4.
2. `VERBOSE=1` é uma opção do make, que aumenta a quantidade de mensagens impressas. Em particular, imprime todos os comandos de compilação com gcc, que podem ser grandes e poluir demais o terminal. Não há problema em desabilitá-la com `VERBOSE=0`.

6.2.1 Versão para a Análise Léxica: `compLabAlunoLEX.bash`

Para a Análise Léxica, deve ser utilizado:

```
1 cmake ..  -DDOPARSE=FALSE
2 make lexdiff VERBOSE=1
```

Esta versão foi disponibilizada como `compLabAlunoLEX.bash`. `DOPARSE` é false, e o target é lexdiff (seção 6.5).

6.2.2 compLabProf.bash e compLabProfLEX.bash

Os mesmos scripts com opção extra para indicar o diretório onde estão as fontes. O professor usa para compilar as submissões de diferentes alunos. Para os alunos pode ser útil para compilar versões diferentes separadas por diretorios.

Cuidado: indicar um outro diretório e compilar uma versão diferente pode não apagar saídas ou diffs anteriores, ou mesmo arquivos intermediários da compilação. O script `killbuild` (seção 6.6) apaga tudo para recomeçar a compilação do zero.

6.3 make: default targets

Chamar make sem argumentos executa os targets definidos como default.

Internamente, CMakeLists.txt define um target para compilar o compilador do aluno, e outro para compilar o TinyCompiler. Ambos devem ser compilados pelo comando `make` sem argumentos. Além disso, são executados por default os scripts `runmycmcomp` e `rundiff`, que respectivamente, compilam todos os exemplos, salvando as saídas no diretório `alunooutput`; e executam os diffs entre as saídas do gabarito e do aluno, salvando os `.diff` também no diretório `alunooutput`.

A compilação default gera diff apenas para as saídas completas, de todas as fases da compilação. A seção 6.4 cobre a geração de diffs para cada fase da compilação.

6.4 target ddiff

```
make ddiff
```

O target ddiff executa diff comparando as saídas de cada fase da compilação, *depois de executar todos os targets default* (seção 6.3). É a ferramenta principal para avaliar as saídas para todas as fases a partir da Análise Sintática.

A comparação é feita contra o diretorio `detail`.

6.5 target lexdiff

```
make lexdiff
```

O target lexdiff executa diff comparando as saídas apenas da fase de análise léxica da compilação, *depois de executar todos os targets default* (seção 6.3).

A comparação é feita contra o diretorio `detailonlylex` (veja seção 2).

6.6 killbuild - apaga saídas e arquivos intermediários

```
1 rm -fr build
2 mkdir build
3 rm -fr alunoout
4 mkdir alunoout
5 rm -fr alunodetail
6 mkdir alunodetail
```

Todas as saídas do compilador, bem como os arquivos intermediários gerados por cmake e make, são apagados, e os diretórios recriados. Útil para **recomeçar a compilação do zero**.

Cuidado: compilar uma versão diferente do compilador pode não apagar saídas ou diffs anteriores, e mesmo arquivos intermediários da compilação. O script `killbuild` apaga tudo para recomeçar a compilação do zero.

Se um terminal estiver aberto em um diretório que foi apagado e recriado, ele não 'encontra' o novo diretório por si mesmo, mesmo com o mesmo nome. Nada mais funciona. Deve-se mudar de diretório e reentrar no diretório em questão. Exemplo:
O terminal estava em:
`%~/ctc41/gen/build$`
foi executado o script `killbuild`, mesmo que em outro terminal. Nada muda no terminal, mas comandos não encontram os arquivos, pois o terminal ainda mantém referência a um diretório que não existe mais. Uma solução é voltar ao diretório pai e reentrar no novo diretório:
`%~/ctc41/gen/build$ cd..`
`%~/ctc41/gen$ cd build`
`%~/ctc41/gen/build$`

7 Analisando diffs e arquivos de texto

Em geral, é comum que logs e dados estejam armazenados em arquivos texto. Por exemplo, adminstradores de sistemas e redes frequentemente analisam logs de sistemas operacionais e redes, serviços, acessos, tráfego, usuários. Dados científicos de baixo volume também frequentemente são armazenados em texto para facilitar a leitura por humanos, mesmo que um formato binário permitisse comprimir mais. Portanto, diversas ferramentas de comparação e análise de diferenças entre arquivos de texto foram desenvolvidas, ...



As ferramentas abaixo permitem uma correção rápida e precisa, não apenas para o professor avaliar, mas também para os alunos encontrarem rapidamente erros em todos os exemplos fornecidos. Portanto, utilize-as!

Devido à quantidade de exemplos de código (560 linhas no total), não espero que desenvolver o compilador nesta disciplina sem empregar ferramentas automatizadas de detecção de diferenças seja viável.
Além disso, utilizar as ferramentas apropriadamente permite que as reuniões para apresentação das fases do compilador sejam rápidas e produtivas, focando no necessário.

Não é permitido apresentar verificando saídas uma por uma.

7.1 Python: compare_diffs.py

O programa python `compare_diffs.py`[3], que deve ser executado no diretório pai de alunodetail e detail, compara os diffs da tabela de símbolos (arquivos com sufixo `tab.diff`), coloca cada linha em um set de strings, e compara os sets entre o gabarito e a saída do aluno.

- O script examina apenas os arquivos `tab.diff`, **não** as saídas `tab.txt`. Ele checa se as linhas que começam com '>' e as que começam com '<' são as mesmas. Portanto, só faz sentido executá-lo depois da execução de `make ddiff` (leia §1.3).

Assim, mesmo que a ordem das variáveis na tabela de símbolos seja diferente, a comparação ainda será válida. Veja §4.

7.2 batcat

Pode-se usar `cat`, ou `less`. Mas `batcat` tem vantagens:

- colore a sintaxe de código e diff files.
- mostra numeração de linhas por default.
- permite mostrar e 'scroll' ao mesmo tempo um conjunto de arquivos diferentes definidos por expressão regular.

Exemplos:

```
% batcat alunodetail/*lex*diff
```

Mostra, de uma só vez, todos os arquivos diff referentes à Análise Léxica, para todos os exemplos. É muito útil para mostrar todas as diferenças de uma fase de uma vez.

```
% batcat alunodetail/mdc_*txt
% batcat alunodetail/mdc_*diff
```

Mostram, respectivamente, as saídas e os diffs, para todas as fases de compilação, para o exemplo `mdc.cm`.

7.3 meld (comparador de diretórios)

GUI para comparar aquivos com highlight de diferenças. Permite comparar diretórios inteiros, mostrando com cores diferentes arquivos iguais e diferentes, e arquivos existentes apenas em um lado. Ao clicar em um dos arquivos, mostra as duas versões lado a lado, com highlights dos trechos e linhas diferentes. Exemplo:

```
% meld alunodetail/ detail/
```

1. meld não está disponível em algumas arquiteturas ou SOs (e.g., MAC arch ARM[4]), e pode não funcionar diretamente via WSL. Nesse caso, encontre outro comparador de diretórios, que marque arquivos iguais e diferentes com cores e permite clicar para verificar as diferenças.

[3] `Agradeçam ao Daniel Quintão.`
[4] https://alternativeto.net/software/meld/?platform=mac

7.4 Opções do diff

Esta seção explica quais opções do comando `diff` são utilizadas pelos scripts. O objetivo é facilitar, ignorando diferenças irrelevantes. O seguinte texto foi retirado diretamente do manual (comando `man diff`)

- `-Z --ignore-trailing-space` ignore white space at line end
- `-b --ignore-space-change` ignore changes in the amount of white space
- `-B --ignore-blank-lines` ignore changes where lines are all blank

Macintoch: O comando `diff` não possui a opção `-Z`. No diretório `scripts`, editem todos os scripts para deletar a opção. As outras opções são aceitas.

8 Dicas de Debug

(veja seção 6.2) opção do make `VERBOSE=1` ordena a impressão de mais detalhes sobre cada comando da compilação do compilador.

8.1 printf, segfaults e buffers

A função `printf` envia a string para um buffer, que posteriormente será impresso no terminal. Em caso de *segfault*, o processo pode terminar sem que o buffer seja impresso na tela, dificultando entender até onde o programa executou antes do *segfault*.

A solução é, **apenas durante debug, não em código de produção**, usar `fprintf` e imprimir no `stderr`, que não usa um buffer. Usar `fflush`(), ou imprimir `'\n'` podem forçar a impressão também[5].

No código disponibilizado `lib/log.c`, há um define chamado `DEBUG_LOG_ALWAYS_FLUSH`, normalmente definido como `0`.
As funções de impressão, `pc()`, `pce()`, e `pp()`, caso este define seja modificado para `1`, sempre chamam flush para todos os arquivos de saída, a cada impressão.
Porque não criei uma variável global, ou funções de get e set para esta funcionalidade? Porque não devem fazer flush a cada impressão em código de produção, é ineficiente chamar o terminal a cada palavra impressa. Podem usar, mas lembrem depois de testar com o define desabilitado, pois **o meu código de teste estará com esta funcionalidade desabilitada**.

8.2 Compilação manual:

Fornecer exemplos e um compilador TinyC completo pode facilitar a implementação em relação à não fornecer nada. Mas, como desvantagem, os alunos podem ser tentados a escrever muito código antes de testar, e criar um *segfault hell*. Compilar manualmente permite testar a compilação de exemplos individuais, fora do script de teste.

Quando o compilador é executado por um script, sobre um conjunto de arquivos, as saídas `stdout` e `stderr` não são facilmente visíveis. Mantenho impressões em `stdout` e `stderr` para *ainda permitir execução manual do compilador no terminal*. Os arquivos para cada fase da compilação ainda serão gerados, por default, em `/tmp`, se um segundo argumento não é fornecido. O executável do compilador é gerado no diretório `build`. Portanto, para testar um exemplo, basta:

```
~/gitctc41/gen/build$ ./mycmcomp ../example/mdc.cm
```

Todas as saídas ainda serão impressas no terminal, e em `/tmp` serão gerados os arquivos referentes à cada fase da compilação.

[5] https://stackoverflow.com/questions/9469790/execution-of-printf-and-segmentation-fault

9 Versões

9.1 2608

1. Criado exemplo `ser9` com nomes repetidos de funções e outros exemplos sugeridos pela T25.
2. Criado exemplo `block2_nointerc.cm`. Para separar o teste da intercalação de declarações do teste de blocos de escopo.
3. Criado exemplo `mdc2.txt`, incorporando o binário e expressões artificiais para testar operadores e operadores unário. Este exemplo satisfaz mdc e conversão binária ao mesmo tempo.
   (a) o gabarito possui um bug no resultado de operadores unários, que deve ser corrigido esse ano.
4. Uniformizado o código em `src, TinyFlex` e `TinyGeracaoCodigo`, restringindo as diferenças no código às realmente relevantes. Deve facilitar a transição da primeira fase às subsequentes.
5. gabarito trocado. Compila com sucesso exemplos com escopo e operador unário. Há bug no resultado da execução do operador unário.
   (a) Escolhi-o como gabarito devido aos bons comentários no código gerado. Talvez sejam úteis.

9.2 2508

1. Criado define no código de log para realizar flush dos arquivos de saída (seção 8.1).
2. Exemplo sort_badchar renomeado para invalid_ch_sort, para deixar claro que se trata de erro léxico.
3. exemplos block2 e block3 (ainda não tratado pelo gabarito)

9.3 240814

Revisão geral de scripts, ferramentas, novo gabarito.

9.3.1 Novo exemplo: ser8_empty_return.cm

1. Uma função declarada com valor de retorno inteiro que termina com 'return;' sem retornar nada.
2. Como o erro depende do tipo da função, não é necessário detectar o erro na Análise Sintática, ele pode ser detectado apenas depois da construção da Tabela de Símbolos.

9.4 231020_1503 log: comentarios e doxgen manual

- Gerada documentação doxygen das funções de log
  - `log.c` e `log.h` incluem comentários em formato doxygen
  - Doxyfile adicionado em `/lib`.
- Atualizadas instruções de compilação manual do TinyCompiler, integrado à minha infraestrutura.
  - `readme.txt`

9.5 230905_1458 Bugfix - msg impressas 2x em ERR

conforme encontramos na aula, as mensagens de erro eram impressas duas vezes nos arquivos ERR.txt portanto foram modificados os arquivos:

- `log.c`, função `pce`, retirando a linha duplicada.
- todos os arquivos `*_err.txt` em `/detail`, refletindo um novo gabarito sem as linhas duplicadas.